CIS 5500: Database and Information Systems
Homework 5: Query Implementation

April 17, 2026

Mustafa Rashid

Spring 2026

---

1. **Question 1: Relational Algebra and Equivalences (15 points)**

   A. (5 points)

   Ans:

   $$\pi_{\text{SName}}(\sigma_{\text{S.SId=E.SId}\wedge\text{S.Age¡21}\wedge\text{E.CId=C.CId}\wedge\text{C.Instructor=“Dr.Naik”}}(S \times E \times C))$$

   B. **5 points**

   Ans:

   $$\pi_{\text{SName}}(\sigma_{\text{S.Age¡21}\wedge\text{C.Instructor=“Dr.Naik”}}((S \bowtie E) \bowtie C))$$

   C. (5 points)

   Ans:

   $$\pi_{\text{SName}}(\pi_{\text{SName, CId}}(\pi_{\text{SId, SName}}(\sigma_{\text{Age¡21}}S) \bowtie \pi_{\text{SId, CId}}E) \bowtie \pi_{\text{CId}}(\sigma_{\text{Instructor=“Dr.Naik”}}C))$$

2. **Question 2: Cost estimation (40 points)**

   A. (5 points)

   Ans: The Researcher relation has 8000 tuples. Out of those $\frac{1}{50}$ are in the field of biology and $\frac{40-12}{40}$ have been active for 12 or more years. So we can estimate that $\frac{1}{50} \cdot \frac{7}{10} \cdot 8000 = 112$ tuples to be the result of the subquery.

B. (10 points)

Ans:

1. The subquery $\sigma_{\text{region}=1\vee\text{region}=3}$Conference will have $\frac{1}{4}\cdot 2000+\frac{1}{4}\cdot 2000 = 1000$ tuples. Each of the 112 filtered researchers has a `cid` that references one of the 2000 conferences uniformly. The probability that a researcher's conference is in region 1 or 3 is $\frac{1000}{2000} = \frac{1}{2}$, so the join yields $112 \cdot \frac{1}{2} = 56$ tuples. Projecting onto two attributes gives $56 \cdot 2 = 112$ attributes, so we need $\lceil \frac{112}{400} \rceil$ or one page.

2. The subquery $\sigma_{\text{region}=4}$Conference will have $\frac{1}{4} \cdot 2000 = 500$ tuples. We then join on `cid` which will give us 500 tuples as each conference has corresponding stats. Projecting this onto 3 attributes will give us 1500 attributes meaning that we will need $\lceil \frac{1500}{400} \rceil$ or 4 pages.

C. (10 points)

Ans:

1. Because |ConfStats| = |Conference| it does not matter which relation we use as outer. The cost will be $20 + \lceil \frac{20}{16} \rceil \cdot 20 = 60$ I/Os.

2. Because |Conference| < |Researcher| we use Conference as the outer relation. The cost will be $20 + \lceil \frac{20}{16} \rceil \cdot 80 = 180$ I/Os.

D. (5 points)

Ans:

1. **BNLJ:** Use Conference as outer (20 pages < 80 pages). Cost $= 20+\lceil \frac{20}{16} \rceil \cdot 80 = 20 + 160 = 180$ I/Os.

2. **Sort-merge join:** Researcher is already sorted on `cid` (cost = 0). Sort Conference: $2(20)(1 + \lceil \log_{17} \lceil \frac{20}{18} \rceil \rceil) = 40(1 + 1) = 80$ I/Os. Merge: $20 + 80 = 100$ I/Os. Total: $80 + 100 = 180$ I/Os.

Both cost 180 I/Os; we prefer sort-merge since it is less sensitive to data skew and the result is sorted.

E. (10 points)

Ans: Total SMJ cost $= 2 \cdot 2X\left(1 + \left\lceil \log_5 \left\lceil \frac{X}{6} \right\rceil \right\rceil\right) + 2X \leq 14X$. Dividing by X and simplifying:

$$\left\lceil \log_5 \left\lceil \frac{X}{6} \right\rceil \right\rceil \leq 2 \implies \left\lceil \frac{X}{6} \right\rceil \leq 25 \implies X \leq 150.$$

The maximum value of X is **150**.

3. **Question 3: Improving queries using EXPLAIN PLAN (45 points)**

A. (15 points) Writing better queries

1. (6 points)

Ans:

(a) **Unnecessary table join:** `LEFT JOIN movie_genres mg` is never referenced in `SELECT` or `WHERE`. It produces duplicate rows for multi-genre movies, forcing a `GROUP BY` to deduplicate. Removing it eliminates the join entirely and the need for grouping.

(b) **Redundant correlated subquery condition:** Inside the `actor_count` subquery, `AND c.movie_id IN (SELECT movie_id FROM cast_in)` is always true since `c` is already drawn from `cast_in`. This forces an unnecessary full scan of `cast_in` on every outer row and should be dropped.

(c) **Inefficient `ALL` quantifier:** `m.rating >= ALL (SELECT rating FROM movies WHERE release_year < 2000)` re-evaluates the inner query per row. Replacing it with `m.rating = (SELECT MAX(rating) FROM movies WHERE release_year < 2000)` is semantically equivalent but computes the maximum once as a scalar.

2. (4 points)

Ans:

```
SELECT m.id, m.title, m.rating,
    (SELECT COUNT(DISTINCT c.person_id) FROM cast_in c
     WHERE c.movie_id = m.id) AS actor_count
FROM movies m
WHERE m.release_year < 2000
AND m.rating = (SELECT MAX(rating) FROM movies WHERE
    release_year < 2000)
AND m.id NOT IN (SELECT c.movie_id FROM cast_in c
```

```
                    JOIN people p2 ON c.person_id = p2.id
                    WHERE p2.name = 'Tom Hanks')
ORDER BY actor_count DESC, m.title;
```

```
QUERY PLAN
Sort  (cost=8293812.82..8293813.72 rows=359 width=43) (actual time=238.039..238.155 rows=6 loops=1)
  Sort Key: ((SubPlan 1)) DESC, m.title
  Sort Method: quicksort  Memory: 25kB
  InitPlan 2 (returns $3)
    ->  Aggregate  (cost=5098.58..5098.59 rows=1 width=32) (actual time=32.037..32.037 rows=1 loops=1)
          ->  Index Only Scan using movies_release_year_and_rating on movies  (cost=0.42..4739.31 rows=143708 width=12) (actual time=0.025..18.165 rows=145382 loops=1)
                Index Cond: (release_year < 2000)
                Heap Fetches: 0
  ->  Index Scan using movies_release_year_and_rating on movies m  (cost=7693.60..8288698.99 rows=359 width=43) (actual time=112.202..238.129 rows=6 loops=1)
        Index Cond: ((release_year < 2000) AND (rating = $3))
        Filter: (NOT (hashed SubPlan 3))
        SubPlan 1
          ->  Aggregate  (cost=23051.54..23051.55 rows=1 width=8) (actual time=24.038..24.038 rows=1 loops=6)
                ->  Index Only Scan using cast_in_pkey on cast_in c  (cost=0.43..23051.53 rows=7 width=4) (actual time=23.308..24.020 rows=0 loops=6)
                      Index Cond: (movie_id = m.id)
                      Heap Fetches: 0
        SubPlan 3
          ->  Nested Loop  (cost=1000.43..7693.17 rows=3 width=4) (actual time=0.388..51.498 rows=63 loops=1)
                ->  Gather  (cost=1000.00..7684.14 rows=1 width=4) (actual time=0.366..51.458 rows=1 loops=1)
                      Workers Planned: 2
                      Workers Launched: 2
                      ->  Parallel Seq Scan on people p2  (cost=0.00..6684.04 rows=1 width=4) (actual time=19.270..34.841 rows=0 loops=3)
                            Filter: ((name)::text = 'Tom Hanks'::text)
                            Rows Removed by Filter: 137090
                ->  Index Only Scan using cast_in_pkey on cast_in c_1  (cost=0.43..8.81 rows=22 width=8) (actual time=0.019..0.030 rows=63 loops=1)
                      Index Cond: (person_id = p2.id)
                      Heap Fetches: 0
Planning Time: 0.605 ms
Execution Time: 238.236 ms
```

3. (5 points)

Ans: Total cost: Andy's query has an estimated total cost of $\approx 2.12 \times 10^9$ compared to $\approx 8.29 \times 10^6$ for the improved query, roughly a $250\times$ reduction. The dominant cost in Andy's plan is the correlated `actor_count` subquery containing a redundant `Semi Join` on `cast_in`, and the `>= ALL` subquery materialized per row. The improved query computes `MAX(rating)` once as an `InitPlan` (actual time 33ms) and pushes both `release_year < 2000` and `rating = $3` into the index condition on `movies_release_year_and_rating`, eliminating the full index scan for the rating filter.
Estimated rows: Andy's plan estimates 35,927 rows in the result; the improved plan estimates 359. The improved estimate is much tighter because the `MAX` scalar is resolved before planning, allowing the optimizer to use selectivity on `rating = $3`.
Indexes used: Both plans use `movies_release_year_and_rating` and `cast_in_pkey`. The improved plan additionally uses `movies_release_year_and_rating` for the `MAX` aggregate via an index-only scan (0 heap fetches), and eliminates the `genres_pkey` index scan entirely by removing the unnecessary `movie_genres` join.
Actual rows returned: 6 rows.

B. Understanding indexes (20 points)

1. (5 points)

Ans:

(a) **Key index:** `movies_pkey` on column `id` (unique).

(b) **Non-key indexes:**

- `movies_num_ratings`: (`num_ratings`)
- `movies_release_year_and_rating`: (`release_year`, `rating`)
- `movies_title_and_runtime`: (`title`, `runtime`)

2. (15 points)

Ans:

(a) **Query:** `SELECT release_year FROM movies WHERE title = 'The Matrix'`
Index used: `movies_title_and_runtime` on `(title, runtime)`.
Comparison: Without the index, Postgres performs a parallel sequential scan over all 395K rows (actual time 30.8ms). With the index, Postgres uses an Index Scan (actual time 0.05ms, ≈600× faster). The index matches the equality search `title = 'The Matrix'` since `title` is a prefix of the index's search key `(title, runtime)`. Since `release_year` is not part of the index search key, Postgres must follow the record ID pointer from the index data entry to retrieve the full record, so an index-only scan is not possible.

```
-- With index
Index Scan using movies_title_and_runtime on movies (cost=0.42..8.44 rows=1 width=4) (actual
      time=0.031..0.032 rows=1 loops=1)
  Index Cond: ((title)::text = 'The Matrix'::text)
Execution Time: 0.049 ms

-- Without index
Gather (cost=1000.00..6568.41 rows=1 width=4) (actual time=7.152..30.749 rows=1 loops=1)
  -> Parallel Seq Scan on movies (cost=0.00..5568.31 rows=1 width=4) (actual time=9.701..14.768 rows=0
       loops=3)
       Filter: ((title)::text = 'The Matrix'::text)
Execution Time: 30.776 ms
```

(b) **Query:** `SELECT rating FROM movies WHERE num_ratings > 100000`
Index used: `movies_num_ratings` on `(num_ratings)`.
Comparison: Without the index, a parallel sequential scan takes 45.4ms to return 2458 rows. With the index, Postgres uses a Bitmap Index Scan followed by a Bitmap Heap Scan (actual time 3.2ms, ≈14× faster). The index matches the range search `num_ratings > 100000` since `num_ratings` is the index's search key. A bitmap scan is chosen over a plain index scan because many rows match; batching record ID lookups in physical page order is more efficient than following each record ID pointer individually.

```
-- With index
Bitmap Heap Scan on movies (cost=31.35..3403.00 rows=2442 width=12) (actual time=0.370..3.049 rows=2458
      loops=1)
  -> Bitmap Index Scan on movies_num_ratings (cost=0.00..30.74 rows=2442 width=0) (actual time=0.209..0.209
       rows=2458 loops=1)
      Index Cond: (num_ratings > 100000)
Execution Time: 3.181 ms

-- Without index
Gather (cost=1000.00..6812.51 rows=2442 width=12) (actual time=0.696..45.285 rows=2458 loops=1)
  -> Parallel Seq Scan on movies (cost=0.00..5568.31 rows=1018 width=12) (actual time=0.151..31.041
       rows=819 loops=3)
      Filter: (num_ratings > 100000)
Execution Time: 45.419 ms
```

(c) **Query:** `SELECT COUNT(*)FROM movies WHERE release_year BETWEEN 2000 AND 2005`
Index used: `movies_release_year_and_rating` on `(release_year, rating)`.
Comparison: Without the index, a parallel sequential scan with aggregation takes 46.9ms. With the index, Postgres uses an Index Only Scan (actual time 41.3ms). The index matches the range search on `release_year` since it is the leading column of the index's search key `(release_year, rating)`.

This scan type differs from queries (i) and (ii) because of what data the query requires. In query (i), `release_year` is not part of the index search key, so Postgres must follow each record ID pointer from the index data entry to the full data record to retrieve it. In query (ii), many rows match a range predicate, so Postgres builds a bitmap of matching record IDs and fetches the corresponding data pages in physical order. In query (iii), `COUNT(*)` requires no column values at all beyond what is already stored in the B+ tree leaf pages. Since `release_year` is in the index search key, the leaf pages alone contain all the information needed, so no record ID pointer is followed (0 heap fetches). This is the defining property of an index-only scan: the index is self-contained for the query, eliminating all data page I/Os. The speedup is modest since 27,918 rows match and all index leaf pages must still be scanned.

```
-- With index
Aggregate (cost=1095.30..1095.31 rows=1 width=8) (actual time=41.262..41.263 rows=1 loops=1)
  -> Index Only Scan using movies_release_year_and_rating on movies (cost=0.42..1023.42 rows=28750 width=0)
       (actual time=0.043..39.036 rows=27918 loops=1)
      Index Cond: ((release_year >= 2000) AND (release_year <= 2005))
      Heap Fetches: 0
Execution Time: 41.307 ms

-- Without index
Finalize Aggregate (cost=7010.54..7010.55 rows=1 width=8) (actual time=46.762..46.833 rows=1 loops=1)
  -> Parallel Seq Scan on movies (cost=0.00..5980.38 rows=11979 width=0) (actual time=0.610..26.986
       rows=9306 loops=3)
      Filter: ((release_year >= 2000) AND (release_year <= 2005))
Execution Time: 46.866 ms
```

C. (10 points)

Ans: Proposed index: a B+ tree index on `people(name)`.

Andy's query contains the subquery `AND m.id NOT IN (SELECT c.movie_id FROM cast_in c JOIN people p2 ON c.person_id = p2.id WHERE p2.name = 'Tom Hanks')`. The EXPLAIN PLAN for Andy's query shows a Parallel Sequential Scan over all 411K rows of `people` filtered by `name = 'Tom Hanks'`, since no index on `name` exists. An index on `people(name)` would match this equality search since `name` is the index's search key, allowing Postgres to locate the matching record via a direct B+ tree lookup rather than scanning the entire relation. This would eliminate the most expensive scan in the NOT IN subquery and reduce the cost of every execution of that subplan.